Proyecto Final Análisis de Sistemas II

Owen Geovany Ligorría Hengstenberg 4090-23-4382

1. Planteamiento del Problema

En la actualidad, la gestión de soporte técnico en diversas organizaciones sufre de descentralización y falta de seguimiento. Las solicitudes de los usuarios suelen ingresar por canales informales (correos directos, mensajes de texto o llamadas), lo que provoca pérdida de información, tiempos de respuesta prolongados y la imposibilidad de medir el rendimiento del equipo de soporte.

Para resolver esta problemática, el presente proyecto implementa un Sistema de Help Desk Automatizado. Esta solución centraliza la ingesta de tickets a través de una interfaz web intuitiva, procesa las solicitudes en tiempo real mediante un motor de automatización (n8n) y garantiza la persistencia y actualización de los estados en una base de datos relacional (PostgreSQL). Esto elimina los cuellos de botella, estandariza el soporte y provee métricas en tiempo real.

2. Arquitectura Local

El sistema está construido bajo un enfoque de microservicios de ejecución local, orquestados mediante contenedores. La arquitectura se divide en tres capas principales:

- **Capa de Presentación (Frontend):** Interfaces desarrolladas en HTML5 y JavaScript Vanilla (index.html y admin.html). Proveen el formulario de ingesta y un panel de control interactivo con capacidades de búsqueda asíncrona y cálculo de KPIs.
- **Capa de Orquestación (Middleware):** Implementada con **n8n**. Actúa como el puente lógico del sistema. Recibe peticiones HTTP (Webhooks) desde el frontend, ejecuta las reglas de negocio (validación de datos, enrutamiento, manejo de errores) y se comunica con la base de datos.
- **Capa de Persistencia (Backend):** Servidor de base de datos **PostgreSQL** contenerizado. Garantiza la integridad de los registros mediante el modelo relacional.
- **Infraestructura:** Todo el backend está virtualizado en **Docker**, definido en un archivo docker-compose.yml, lo que asegura que el entorno sea replicable, aislado y agnóstico al sistema operativo host.

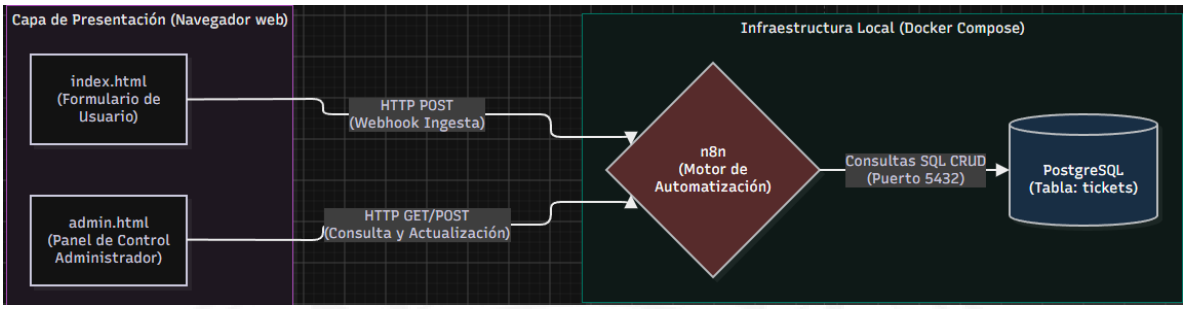
3. Modelo de Datos

La persistencia de la información se gestiona en PostgreSQL mediante una estructura simplificada y eficiente para el ciclo CRUD.

Tabla: tickets

- id (SERIAL, PRIMARY KEY): Identificador único autoincremental del ticket.
- correo_usuario (VARCHAR): Correo electrónico del solicitante para futuras notificaciones.
- asunto (VARCHAR): Título descriptivo del problema.
- descripcion (TEXT): Detalles completos de la solicitud de soporte.
- estado (VARCHAR): Rastrea la etapa del ticket. Por defecto se inicializa como 'Nuevo', y muta a 'Resuelto' a través de la intervención del administrador.

4. Diagrama de Arquitectura



5. Descripción de Workflows:

El núcleo operativo del Help Desk recae en 4 flujos de trabajo principales configurados en n8n, los cuales actúan como microservicios independientes:

- **Workflow 1 - Ingesta de Tickets:** Este flujo se activa mediante un nodo Webhook que escucha las peticiones POST provenientes de index.html. Al recibir el JSON con los datos del usuario, ejecuta un nodo de PostgreSQL configurado con una sentencia INSERT, guardando el asunto, descripción y correo, e inicializando el estado del ticket como "Nuevo". Finalmente, retorna un código HTTP 200 al frontend para confirmar la recepción.
- **Workflow 2 - Consulta de Tickets:** Diseñado para alimentar el panel de control del administrador. Un Webhook GET dispara una consulta SELECT * FROM tickets en la base de datos. Los resultados se devuelven al frontend (admin.html) en formato JSON, permitiendo que JavaScript renderice dinámicamente la tabla y las tarjetas de KPIs.
- **Workflow 3 - Actualización de Estado (Admin):** Permite cambiar el estado de un ticket específico. Recibe a través de un webhook el ID del ticket y ejecuta una sentencia UPDATE tickets SET estado = 'Resuelto' WHERE id = $1. Esto cierra el ciclo de soporte de manera instantánea.

- **Workflow 4 - Notificaciones (Opcional/Escalabilidad):** Flujo de soporte preparado para el envío de alertas mediante servicios externos, garantizando que los eventos críticos del sistema generen notificaciones asíncronas sin retrasar la respuesta de los demás webhooks.

6. Manejo de Errores y Tolerancia a Fallos

El sistema implementa una estrategia de control de excepciones distribuida en múltiples capas para garantizar que una petición malformada o una desconexión no provoque la caída total de la aplicación:

- **Validación en el Frontend (Cliente):** La interfaz web de ingesta de tickets (index.html) cuenta con validaciones de campos obligatorios en los formularios antes de realizar la petición HTTP. Adicionalmente, las llamadas asíncronas (fetch) están envueltas en bloques try...catch que interceptan problemas de red (como la caída del servidor local) y muestran alertas amigables al usuario sin romper la interfaz gráfica.
- **Gestión de Excepciones en n8n (Middleware):** Los flujos de trabajo están diseñados para no detenerse ante errores de ejecución. Si un nodo de PostgreSQL falla (por ejemplo, por un tipo de dato incorrecto o un ID de ticket inexistente), los flujos están configurados para retornar una respuesta HTTP 400 o 500 al cliente, indicando la naturaleza del error de forma estructurada.
- **Resiliencia del Motor de Base de Datos:** PostgreSQL garantiza las propiedades ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad). Si una transacción de actualización de un ticket es interrumpida abruptamente, el motor revierte los cambios para mantener la consistencia del registro original.

7. Ciberseguridad y Consideraciones de Protección

Al tratarse de una arquitectura orientada a microservicios, se adoptaron prácticas estándar de la industria para proteger las credenciales y el flujo de la información:

- **Aislamiento de Credenciales (Variables de Entorno):** Se eliminó por completo el uso de credenciales expuestas en texto plano (Hardcoding). Las contraseñas de la base de datos, el usuario y la configuración de red se gestionan exclusivamente a través de un archivo .env externo. Los servicios en el archivo docker-compose.yml leen estas variables dinámicamente durante el arranque.
- **Prevención de Fugas de Información (Git):** Para garantizar que el repositorio público sea seguro, se configuró una política de exclusión mediante el archivo .gitignore. Esto asegura que el archivo .env jamás sea indexado ni subido a los servidores de GitHub, protegiendo la infraestructura de posibles atacantes.
- **Aislamiento de Redes Internas (Docker Networks):** Los contenedores de n8n y PostgreSQL operan dentro de una red privada virtualizada creada por Docker Compose. La base de datos no expone puertos innecesarios a la internet pública; únicamente n8n tiene el permiso y la visibilidad para comunicarse con PostgreSQL de manera local, reduciendo drásticamente la superficie de ataque.

8. Especificación de Integración (Webhooks)

El sistema utiliza un enfoque de API REST simulada a través de los nodos Webhook de n8n. A continuación, se detalla la estructura de los *cargas útiles (payloads)* para la comunicación entre el Frontend y el Middleware.

8.1. Endpoint de Ingesta (Creación de Ticket)

- **Método:** POST
- **Función:** Recibe los datos del formulario de usuario y los inserta en PostgreSQL.
- **Estructura del Body (JSON):**

JSON

{

"correo_usuario": "estudiante@umg.edu.gt",

"asunto": "Problema de acceso al portal",

"descripcion": "No puedo ingresar con mis credenciales actuales."

}

- **Respuesta Exitosa (200 OK):** {"mensaje": "Ticket creado exitosamente"}

8.2. Endpoint de Consulta (Lectura de Tickets)

- **Método:** GET
- **Función:** Retorna la lista completa de tickets para alimentar la tabla y los KPIs del panel de control.
- **Respuesta Exitosa (200 OK):** Arreglo de objetos JSON con la estructura completa de la tabla tickets.

8.3. Endpoint de Actualización (Resolución de Ticket)

- **Método:** POST (o PATCH)

- **Función:** Actualiza el estado de un ticket específico a 'Resuelto'.

- **Estructura del Body (JSON):**

JSON

{

"id": 15

}

9. Casos de Prueba (Testing)

Para validar la resiliencia del sistema, se ejecutaron pruebas funcionales basadas en los requerimientos del proyecto:

1. **Caso de Prueba 1 (Happy Path - Flujo Normal):** Se ingresó un ticket desde la interfaz de usuario con todos los campos completos. El sistema retornó un código 200, el registro apareció inmediatamente en el panel admin.html con estado "Nuevo", y los contadores de KPIs se actualizaron correctamente sumando 1 al total.
2. **Caso de Prueba 2 (Manejo de Errores - Datos incompletos):** Se forzó el envío de una petición HTTP al webhook de ingesta sin el campo "correo_usuario". El sistema de n8n interceptó la anomalía, el nodo de base de datos rechazó la transacción por violar la restricción de nulidad, y el flujo retornó un error controlado sin colapsar el contenedor Docker.
3. **Caso de Prueba 3 (Actualización de Estado):** Desde el panel de administrador, se hizo clic en la acción "Resolver" de un ticket existente. El sistema envió el ID correctamente, la base de datos mutó el estado, y la interfaz se recargó asíncronamente mostrando el ticket en color verde ("Completado").

10. Conclusiones y Trabajo Futuro:

- **Desacoplamiento de Arquitectura:** La implementación de Docker y n8n demostró que separar el frontend, la lógica de negocio y la base de datos permite un desarrollo más ágil y un mantenimiento más seguro, evitando los riesgos de arquitecturas monolíticas.
- **Eficiencia en Soporte Técnico:** El sistema cumple con su objetivo principal de centralizar y automatizar el soporte, eliminando el seguimiento manual y proporcionando métricas en tiempo real a los administradores.
- **Trabajo Futuro (Escalabilidad):** La arquitectura actual deja la puerta abierta para integraciones futuras (Workflow 4), como el envío automático de correos electrónicos al usuario cuando su ticket cambia a estado "Resuelto", o la conexión con bots de Telegram/WhatsApp para soporte omnicanal.